

category: dotnet-general

tags: [git, github, native-binary, ffmpeg, git-history-rewrite, file-size-limit]

severity: high

created: 2026-07-16

updated: 2026-07-16

project-origin: Kztek.Camera (migrate Kztek.Cameras WinForms → Avalonia)

`git push` bị GitHub từ chối vì native binary (.so/.dll) > 100MB — phải rewrite history, không chỉ xóa ở commit mới

Tình huống gặp phải

Đang bổ sung file FFmpeg native `.so` cho Linux

(`avcodec/avformat/avutil/swscale/swresample/avdevice/avfilter`) vào project thư viện Avalonia

(`Kztek.Cameras.Avalonia`) để hoàn thiện cross-platform build. Đã `git add` + `git commit` các file này (mỗi lib có 3 bản: bare + major-version + full-version, byte-identical). Build/publish local đều pass. Vài commit sau, chạy `git push` — lệnh treo/timeout, và IDE (VS Code Git extension) của user hiện popup lỗi thật.

Triệu chứng / Lỗi

```
File size limit exceeded
The push operation includes a file which exceeds GitHub's file size restriction of
100MB.
Files that exceed the limit
• 1.Source/Kztek.Cameras.Avalonia/Players/FFMPEG/Resource/libavcodec.so (100.85 MB)
```

`git push` từ Bash tool không in ra lỗi này trực tiếp — chỉ treo rồi timeout (exit code 143). Lỗi thật chỉ thấy qua UI của IDE khác đang chạy song song.

Nguyên nhân gốc rễ (Root Cause)

GitHub **chặn cứng (hard reject)** mọi single blob > 100MB khi push qua protocol thường (không phải chỉ cảnh báo như ngưỡng 50MB). File `libavcodec.so` build đủ codec/encoder nặng hơn bản Windows tương ứng (`avcodec-61.dll` chỉ 85MB — cùng thư viện nhưng build khác, nhẹ hơn) nên vượt ngưỡng dù trước đó chưa ai để ý kiểm tra kích thước từng file trước khi `git add`.

Quan trọng: **file đã nằm trong lịch sử git (đã commit)** — nên chỉ `git rm` + commit mới KHÔNG đủ, vì blob cũ vẫn tồn tại trong các commit trước đó và vẫn bị đẩy lên khi push (git push toàn bộ object mới trên remote, kể cả object nằm trong commit cũ chưa từng lên remote).

Giải pháp

Vì repo local **chưa từng push thành công** lần nào trước đó (`git ls-remote` xác nhận remote vẫn ở commit rất cũ), sửa lịch sử local an toàn — không ảnh hưởng ai khác:

```
# 1. Stash MỌI thay đổi chưa commit trước khi rewrite (kể cả untracked, dùng -u)
git stash push -u -m "WIP trước khi rewrite history"

# 2. Xóa file khỏi TẤT CẢ commit trên nhánh (không phải chỉ commit mới nhất)
git filter-branch --force --index-filter \
  'git rm -r --cached --ignore-unmatch "path/to/big1.so*" "path/to/big2.so*" ' \
  -- <branch-name>

# 3. Dọn backup ref filter-branch tự tạo + gc để blob thật sự biến mất khỏi .git
git for-each-ref --format='% (refname)' refs/original/ | xargs -r -n1 git update-ref -d
rm -rf .git/refs/original
git reflog expire --expire=now --all
git gc --prune=now --aggressive

# 4. Verify blob đã biến mất
git rev-list --objects --all | git cat-file --batch-
check='% (objecttype) % (objectname) % (objectsize) % (rest)' | sort -k3 -n -r | head

# 5. Khôi phục lại thay đổi đã stash
git stash apply stash@{0}
# verify nội dung khớp trước khi drop
git stash drop stash@{0}
```

1. Thêm `.gitignore` cho pattern file native lớn để tránh lỡ commit lại.
2. Cập nhật csproj: dùng `<None Include="...*.so*" Condition="Exists('...')">` — chỉ copy nếu file có mặt cục bộ, không bắt buộc phải có trong repo (giống pattern reference pre-built DLL ngoài repo: `KztekComponentAvalonia`).

Áp dụng lại (How to reuse)

- **TRƯỚC KHI `git add` bất kỳ file binary native nào (.so, .dll, .exe build riêng) →** `ls -la` hoặc `du -sh` kiểm tra kích thước từng file. Nếu > 50MB → cân nhắc kỹ trước khi commit (GitHub cảnh báo ở 50MB, chặn cứng ở 100MB).
- Nếu phát hiện file quá lớn đã LỖ commit (chưa push) → dùng quy trình `filter-branch` + stash ở trên. **KHÔNG** chỉ `git rm` + commit mới (không xóa được blob khỏi lịch sử).
- Trước khi rewrite history, **LUÔN** `git ls-remote origin <branch>` để xác nhận remote có commit nào của mình chưa — nếu remote ĐÃ có, rewrite history sẽ cần force-push và ảnh hưởng người khác đang pull nhánh đó (nguy hiểm hơn nhiều, cần user xác nhận rõ ràng riêng).
- File native lớn có nhiều bản trùng tên (bare + version suffix, y hệt byte-for-byte) — `git dedupe` theo content hash nên không tốn thêm dung lượng blob, nhưng vẫn phải để ý **TỪNG** bản riêng lẻ nếu > 100MB (GitHub check theo blob, filename nào cũng bị chặn như nhau).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `git filter-branch --index-filter` chỉ rewrite ref được chỉ định (VD `feat/avalonia`) — **KHÔNG** tự động dọn `refs/stash`. Nếu đã `git stash` TRƯỚC KHI rewrite, stash đó vẫn giữ tham chiếu tới lịch sử CŨ (commit chưa rewrite) → blob nặng vẫn "sống" qua stash dù đã `filter-branch` + `gc`. Phải xóa `refs/stash` (sau khi đã `apply/verify` nội dung an toàn) rồi `gc` lại mới thật sự sạch.
- ⚠ `git stash drop` là hành động phá hủy không hoàn tác — trong harness có safety classifier, thao tác này có thể bị chặn nếu chưa được user xác nhận rõ ràng bằng lời (không tự suy luận từ ngữ cảnh). Luôn hỏi lại trước khi drop, dù đã `verify` nội dung an toàn.

- ⚠️ `dotnet run/dotnet build` không phải công cụ để verify P/Invoke resolver hoạt động — phải viết console app scratch thật sự GDI hàm PInvoke để xác nhận load được native lib, build pass không có nghĩa là load runtime đúng.
- ⚠️ Sau `git gc --aggressive` trên repo lớn (nhiều native binary khác vẫn còn, VD ReadPL.dll 60MB, avcodec-61.dll 89MB) có thể mất thời gian đáng kể — không phải dấu hiệu treo/lỗi.

Tham chiếu

- Project liên quan: `Kztek.Camera` — [.claude/plans/PLAN-kztek-cameras-to-avalonia-2026-07-13.md](#)
- GitHub file size limits: <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github>